

Word Embeddings: Word2Vec, GloVe, FastText

Lecture 8: The Neural Revolution in NLP

LLM Course - Week 3

Winter 2026

Today's Lecture ?

1. ? The 2013 Revolution: Word2Vec
2. ? Word2Vec Architectures: CBOW & Skip-gram
3. ? GloVe: Global Vectors
4. ? FastText: Subword Information
5. ? Comparison & Evaluation
6. ? Applications & Best Practices

Goal: Understand how neural methods learn dense word representations

Count-Based (LSA, LDA):

- Build co-occurrence matrix
- Apply matrix factorization
- Global statistics
- Linear relationships
- Interpretable factors

Advantages:

- Fast training
- Leverages global statistics
- Mathematically grounded

From Count-Based to Prediction-Based ?

Count-Based (LSA, LDA):

- Build co-occurrence matrix
- Apply matrix factorization
- Global statistics
- Linear relationships
- Interpretable factors

Advantages:

- Fast training
- Leverages global statistics
- Mathematically grounded

Prediction-Based (Word2Vec):

- Predict context from word
- Train neural network
- Local context windows
- Non-linear relationships
- Learned representations

Advantages:

- Better semantic capture
- Scalable to huge corpora
- Dense, low-dimensional

2013: Word2Vec showed prediction-based methods could outperform

2013: Everything changed

Key Innovation:

- Shallow neural network
- Predict context from word (CBOW)
- Predict word from context (Skip-gram)
- Dense, low-dimensional vectors
- Captures semantic relationships
- **Fast** training with negative sampling

Famous Results:

$$\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$$

Other Examples:

- $\vec{Paris} - \vec{France} + \vec{Italy} \approx \vec{Rome}$
- $\vec{walking} - \vec{walk} + \vec{swim} \approx \vec{swimming}$
- $\vec{bigger} - \vec{big} + \vec{small} \approx \vec{smaller}$

Vector arithmetic captures semantic relationships!

Words that appear in similar contexts should have similar representations

Example Context Window

"The quick brown [TARGET] jumped over the lazy dog"

Context: [The, quick, brown, jumped, over, the, lazy, dog]

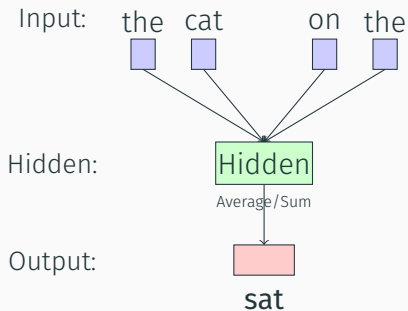
Target: fox

Key Idea:

- Train a model to predict target from context (or vice versa)
- The learned **hidden layer weights** become our word vectors!

CBOW: Continuous Bag of Words ?

Predict the center word from context words



Architecture:

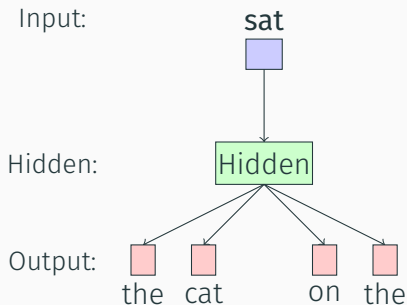
1. Input: One-hot vectors of context words
2. Hidden: Average of input embeddings
3. Output: Softmax over vocabulary

Training:

- Context window size (e.g., 5 words)
- Maximize $P(\text{center}|\text{context})$
- Backpropagation updates embeddings

Skip-gram: The Inverse Approach ?

Predict context words from the center word



Architecture:

1. Input: One-hot vector of center word
2. Hidden: Word embedding (lookup)
3. Output: Softmax over vocabulary ($\times C$ times)

Training:

- For each context position
- Maximize $P(\text{context}|\text{center})$
- Independent predictions

Negative Sampling: The Speed Trick ?

Problem: Softmax over entire vocabulary is too expensive!

$$P(w_O|w_I) = \frac{\exp(v_{w_O}^T v_{w_I})}{\sum_{w=1}^V \exp(v_w^T v_{w_I})}$$

For 100k vocabulary: Need to compute 100k exponentials per training example! ?

Negative Sampling: The Speed Trick ?

Problem: Softmax over entire vocabulary is too expensive!

$$P(w_O|w_I) = \frac{\exp(v_{w_O}^T v_{w_I})}{\sum_{w=1}^V \exp(v_w^T v_{w_I})}$$

For 100k vocabulary: Need to compute 100k exponentials per training example! ?

Solution: Negative Sampling

Instead of predicting across all words, create a binary classification:

- **Positive sample:** Actual context word (label = 1)
- **Negative samples:** K random words (label = 0)

Word2Vec in Practice ?

```
from gensim.models import Word2Vec
import gensim.downloader as api

# Load pre-trained model (300 dimensions, 3B words)
model = api.load('word2vec-google-news-300')

# Find similar words
similar = model.most_similar('computer', topn=5)
print(similar)
# Output: [('computers', 0.72), ('laptop', 0.69), ('PC', 0.68),
...]
```



```
# Word analogies: king - man + woman = ?
result = model.most_similar(
    positive=['king', 'woman'],
    negative=['man'],
```

GloVe: Global Vectors for Word Representation [?]

Combining the best of count-based and prediction-based methods

Motivation:

- Word2Vec uses **local** context windows
- LSA uses **global** co-occurrence statistics
- Can we get the best of both?

Key Insight:

Ratios of co-occurrence probabilities encode meaning better than raw probabilities!

Ratio Encodes Relevance:

$$\frac{P(\text{solid}|\text{ice})}{P(\text{solid}|\text{steam})} \gg 1$$

$$\frac{P(\text{gas}|\text{ice})}{P(\text{gas}|\text{steam})} \ll 1$$

$$\frac{P(\text{water}|\text{ice})}{P(\text{water}|\text{steam})} \approx 1$$

Idea: Learn word vectors such that their dot product relates to co-occurrence probability ratios

GloVe: The Objective Function ?

Goal: Learn vectors that capture co-occurrence statistics

Objective Function:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(\vec{w}_i^T \tilde{\vec{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

where:

- X_{ij} = number of times word j appears in context of word i
- $\vec{w}_i$ = word vector for word i
- $\tilde{\vec{w}}_j$ = separate context vector for word j
- $b_i, \tilde{b}_j$ = bias terms
- $f(X_{ij})$ = weighting function

Weighting Function

Word2Vec (Skip-gram):

- Local context windows
- Online training
- Predicts context from word
- Stochastic updates
- Negative sampling trick
- Each occurrence matters

Advantages:

- Works well on small corpora
- Can train incrementally
- Captures local patterns

GloVe:

- Global co-occurrence matrix
- Batch training
- Factorizes co-occurrence matrix
- Deterministic
- Weighted least squares
- Aggregates all occurrences

Advantages:

- Faster training (large corpora)
- Better statistical efficiency
- More interpretable

```
import gensim.downloader as api
import numpy as np

# Load pre-trained GloVe embeddings
# Options: glove-wiki-gigaword-50, -100, -200, -300
# Or: glove-twitter-25, -50, -100, -200
glove = api.load("glove-wiki-gigaword-100")

# Use just like Word2Vec
similar = glove.most_similar('computer', topn=5)
print(similar)

# Analogies
result = glove.most_similar(
    positive=['france', 'berlin'],
    negative=['paris'],
```

The Problem with Word2Vec and GloVe:

Out-of-Vocabulary (OOV) Problem

- Word2Vec/GloVe: One vector per word
- New word? **No vector!**
- Misspelling? **No vector!**
- Rare morphological form? **No vector!**

Example:

- Have: "running", "runner"
- Need: "runnable" → **OOV!**
- But these words share morphology! ("run" + suffix)

FastText: Character N-grams

Key Idea: Break words into character n-grams

Example: "where" (with n=3)

Character trigrams: <wh, whe, her, ere, re>

Plus: <where> (the whole word)

Word vector:

$$\vec{\text{where}} = \frac{1}{6} \left(\langle \vec{\text{wh}} \rangle + \vec{\text{whe}} + \vec{\text{her}} + \vec{\text{ere}} + \vec{\text{re}} + \langle \vec{\text{where}} \rangle \right)$$

Advantages:

- Handles OOV words!
- Captures morphology

Example OOV:

Have trained on: "run", "running"

Need vector for: "runnable"

N-grams: <ru, run, unn, nna, nab, abl,

```
from gensim.models import FastText
import gensim.downloader as api

# Load pre-trained FastText model
# Available in 157 languages!
fasttext_model = api.load('fasttext-wiki-news-subwords-300')

# Works for known words
vec_cat = fasttext_model['cat']

# Also works for unknown words! (OOV)
vec_unknownword = fasttext_model['unknownword'] # Still get a
vector!

# Even works for misspellings (somewhat)
vec_misspelling = fasttext_model['computr'] # Close to "computer"
```

Embedding Methods Comparison

Method	Year	Type	OOV	Speed	Best For
LSA	1990	Count		Medium	IR, exploration
LDA	2003	Probabilistic		Slow	Topic modeling
Word2Vec	2013	Neural (local)		Fast	General NLP
GloVe	2014	Hybrid (global)		Fast	Large corpora
FastText	2017	Neural + ngrams		Fast	Morphology-rich

When to Use What?

- **Word2Vec (Skip-gram):** General purpose, good semantic quality, most popular
- **GloVe:** Large corpus, want deterministic training, good interpretability
- **FastText:** Morphologically rich languages, need OOV handling, many rare words

Evaluating Word Embeddings ?

Intrinsic Evaluation:

1. Word Similarity

- WordSim-353 dataset
- SimLex-999 dataset
- Spearman correlation with human judgments

2. Word Analogies

- Google Analogy Dataset (19k questions)
- Semantic:
Athens:Greece::Baghdad:Iraq
- Syntactic: *good:better::bad:worse*

Extrinsic Evaluation:

Use embeddings as features in downstream tasks:

- Sentiment Analysis
- Named Entity Recognition
- POS Tagging
- Machine Translation
- Question Answering
- Text Classification

Key Insight

Intrinsic scores don't always correlate with extrinsic performance!

Limitations of Static Embeddings [?]

The fundamental problem: One vector per word type

Example: Polysemy

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)
3. "The plane will **bank** left" (tilt)

All get the **same vector**! This conflates different meanings.

Other Limitations:

- No compositional semantics
- "hot dog" $\neq$ "hot" + "dog"
- Bias in embeddings

The Solution:

- **Contextual embeddings!**
- Different vector per occurrence
- Consider sentence context

Embeddings learn the biases in their training data

Gender Bias Examples

- $\vec{\text{man}} : \vec{\text{programmer}} :: \vec{\text{woman}} : \vec{\text{homemaker}}$
- $\vec{\text{father}} : \vec{\text{doctor}} :: \vec{\text{mother}} : \vec{\text{nurse}}$
- "man" more associated with "career", "woman" with "family"

Other Biases:

- Racial/ethnic stereotypes
- Age bias
- Religious bias
- Socioeconomic bias

How can we reduce bias in embeddings?

1. Post-processing:

- Identify bias subspace
- Neutralize: Remove bias from neutral words
- Equalize: Ensure equal distances

2. Training-time:

- Counterfactual data augmentation
- Adversarial debiasing
- Constrained optimization

3. Data curation:

Challenges:

- Hard to define "bias"
- May harm performance
- Bias is multidimensional
- Not a complete solution
- Ethical considerations

Important

Debiasing is an active research area.
No perfect solution yet!

Best practice:

Practical Tips for Using Embeddings

1. Start with pre-trained models

- Word2Vec: Google News (300d, 3B words)
- GloVe: Common Crawl (300d, 840B tokens)
- FastText: Available in 157 languages

2. Fine-tune if you have domain data

- Medical: PubMed, clinical notes
- Legal: case law, contracts
- Social media: tweets, posts
- Can significantly improve performance

3. Choose dimensions wisely

- More dims = more expressiveness, more data needed
- 50-100d: small datasets, fast computation
- 200-300d: large datasets, better quality

4. Use cosine similarity

- $\text{similarity}(u, v) = \frac{u \cdot v}{\|u\| \cdot \|v\|}$
- Not Euclidean distance (direction matters more than magnitude)

Search & Information Retrieval:

- Semantic search
- Query expansion
- Document ranking
- Question answering

Text Classification:

- Sentiment analysis
- Spam detection
- Topic classification
- Intent detection

Sequence Labeling:

- Named entity recognition

Recommendation Systems:

- Product recommendations
- Content recommendations
- User similarity

Machine Translation:

- Word alignment
- Transfer learning
- Multilingual models

Creative Applications:

- Poetry generation
- Analogy discovery

- Semantic exploration

Why do word analogies work so well?

The famous example:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

Think about:

- What does vector subtraction represent linguistically?
- Why does this capture semantic relationships?
- What are the limitations of this approach?
- Does this mean embeddings "understand" gender?

What we learned today:

1. **Word2Vec (2013):** Neural revolution in embeddings
 - CBOW: Predict center from context
 - Skip-gram: Predict context from center
 - Negative sampling for efficiency
2. **GloVe (2014):** Combining count + prediction
 - Global co-occurrence statistics
 - Factorization objective
 - Ratio of probabilities
3. **FastText (2017):** Subword information
 - Character n-grams
 - Handles OOV words
 - Captures morphology
4. **Evaluation:** Intrinsic vs. extrinsic
5. **Limitations:** Static, biased, no polysemy

Foundational Papers:

- Mikolov et al. (2013a). "Efficient Estimation of Word Representations in Vector Space"
- Mikolov et al. (2013b). "Distributed Representations of Words and Phrases and their Compositionality"
- Pennington et al. (2014). "GloVe: Global Vectors for Word Representation"
- Bojanowski et al. (2017). "Enriching Word Vectors with Subword Information"

Bias & Ethics:

- Bolukbasi et al. (2016). "Man is to Computer Programmer as Woman is to Homemaker?"
- Caliskan et al. (2017). "Semantics derived automatically from language corpora contain human-like biases"

Next Lecture:

Contextual Embeddings: ELMo, Universal Sentence Encoder, BERT

One word, multiple meanings!